XRPL Validator — Daily Engineering Report · 2026-07-25

Period: 2026-07-25 00:00 to 24:00 US Eastern (2026-07-25 04:00 to 2026-07-26 04:00 UTC). Generated 2026-10-21 00:47 EDT by report generator v3.0. Sources: the XRP Ledger (ledger headers and amendments, read from a reference rippled node), the validator's signing log and receipt logs, the deployment pipeline's gate, soak and health records, the git history of xrpl-validator and the XRPL Rust SDK, and data.xrpl.org. Scope: XRPL work only.

Revised on 2026-10-21 in report format v3; the original is kept in this repository's history. Built only from records that exist for this day. Available: xrpl-validator merge records (they begin 2026-07-04); data.xrpl.org agreement scores. Not available: ledger headers (the reference node holds ledgers from 2026-10-04 22:52 UTC); signing records (they begin 2026-09-19); Step 1 receipt records (they begin 2026-08-29); Steps 2–5 receipt records (they begin 2026-10-09); deployment records (they begin 2026-09-11); amendment data.

Status

Not assessed — no signing records for this day.

Criteria. Normal: coverage outside planned maintenance at or above the 99.9% objective, every validation agreed with the network's validated ledger, and no engine receipt is open. Degraded: coverage below the objective, or an open engine receipt. Impaired: any validation that disagreed with the network, or coverage below 90%.

Executive summary

- **Validator:** no signing records for this day (they begin 2026-09-19).
- **Native engine:** no receipt records for this day (they begin 2026-08-29).
- **Changes:** 0 deployments, 0 commits merged into main.

Service levels

No signing records for this day (they begin 2026-09-19).

External agreement, data.xrpl.org (UTC day 2026-07-25): 1.00000 (0 missed of 21,647). *data.xrpl.org scores UTC days from the validations its crawlers received.*

Native engine conformance

No receipt records for this day (Step 1 records begin 2026-08-29, Steps 2–5 records 2026-10-09).

Live counters are kept only for the current run; none exist for a past day.

Receipt register

No receipt records for this day (Step 1 records begin 2026-08-29).

Incidents

Not assessed: no signing records for this day.

Change log

Deployments

No deployment records before 2026-09-11.

Merged into main

No merges into main.

Findings fixed

No engine findings merged into main this day.

Other engineering changes

- **fix(xrpl-ledger): cap payment delivery at the destination's trust limit** (`04a1586`, xrpl-validator) The last TER-class divergence cluster (6: 105740164/105741244/105741859, 2 each) was not the two-hop AMM strand bug the handoff suspected. rippled's final delivery step DirectStepI::maxPaymentFlow (DirectStep.cpp:487) limits what a payment can land on the destination to creditLimit(dst,issuer) minus what it already holds, so a destination at or over its own trust limit receives nothing and the whole flow is …
- **fix(xrpl-ledger): cross an underfunded maker at book quality, don't rest** (`84898d4`, xrpl-validator) The section-1 achieved-quality break (offer.rs) was calibrated for the #105780948 IoC tecKILLED, where the TAKER's own side bounds the fill and the floored output realises worse than the limit. It also fired, wrongly, on a plain offer crossing a maker that is merely UNDERFUNDED at a book quality equal to the taker's limit: the <=1e-7 round-up on `pay` for the small maker-funds-clamped fill tips the achieved rate …
- **fix(xrpl-ledger): re-check a maker's actual quality, not just its book page** (`7e592e9`, xrpl-validator) rippled compares a crossing maker's ACTUAL getRate to the taker's limit, not merely the quantized book-directory level it rests in. An offer can sit in a page that ties the threshold while its exact rate is one ULP worse, and rippled leaves such an offer untouched.
- **fix(xrpl-ledger): honour tfPassive -- don't consume equal-quality makers** (`77f6f39`, xrpl-validator) We never read tfPassive: a passive offer was crossed like a normal one. A passive offer crosses only STRICTLY better makers and rests behind equal-quality ones (rippled OfferCreate.cpp:396 `++threshold`). Our rate encoding is inverted (lower = better, monotone in the u64), so tighten the threshold by one ULP for passive offers; the existing q<=threshold and maker-rate gates then exclude the equal level.

- **fix(xrpl-ledger): implement AMMWithdraw tfWithdrawAll teardown** (`a7bd6a0`, xrpl-validator) tfWithdrawAll fell through the partial-withdraw path: it withdrew NO assets (no Amount fields to loop over) and burned a fixed default LPToken amount, leaving the LPToken trust line Modified-at-zero instead of torn down.
- **fix(differential_probe): hydrate the issuer's owner-dir tail for CheckCash** (`7eb0dca`, xrpl-validator) CheckCash creates the delivered IOU's trust line, which the ISSUER co-owns -- but native_read_keys loaded only the casher's owner-dir root, so an issuer with a multi-page directory was never hydrated and dir_insert invented a fresh root instead of appending to the existing last page.

Independence program

No independence record for UTC day 2026-07-25; records begin 2026-10-20.

Network

No ledger records for this day: the reference node holds ledgers from 2026-10-04 22:52 UTC (#107,436,647).

Amendment watch

No amendment records for this day.

Open items

None.

Corrections to earlier reports

None.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Commit-Reveal Batch Auctions for DEX Offers** (dex) Add a tfHidden flag to OfferCreate that stores only sfCommitHash (SHA-256 of taker_pays, taker_gets, account, nonce) instead of plaintext amounts, fil
- **Travel-Rule Payload Requirement for VASP Payments** (compliance) Add an owner-reserved VASPPolicy ledger object (keylet::vaspPolicy(account)), set by a VASPPolicySet transaction, carrying sfTravelRuleThreshold — an
- **Atomic Flash Borrow/Repay for Vault Assets** (lending) Add a VaultFlashBorrow transaction on SingleAssetVault that debits available vault liquidity to the submitting account and records an in-flight sfFlas

Appendix — commits

Committed	Merged into main	Repo	Commit	Subject
07-25 12:37	—	xrpl-validator	`04a1586`	fix(xrpl-ledger): cap payment delivery at the destination's trust limit
07-25 14:38	—	xrpl-validator	`84898d4`	fix(xrpl-ledger): cross an underfunded maker at book quality, don't rest
07-25 17:01	—	xrpl-validator	`7e592e9`	fix(xrpl-ledger): re-check a maker's actual quality, not just its book page
07-25 20:13	—	xrpl-validator	`77f6f39`	fix(xrpl-ledger): honour tfPassive -- don't consume equal-quality makers
07-25 22:57	—	xrpl-validator	`a7bd6a0`	fix(xrpl-ledger): implement AMMWithdraw tfWithdrawAll teardown
07-26 01:02	—	xrpl-validator	`7eb0dca`	fix(differential_probe): hydrate the issuer's owner-dir tail for CheckCash

Times in UTC. A commit appears on the day it was committed to main or merged into it.